LingBot-VLA 2.0 SFT：根单元 AllGather 16 字节对齐

8 张 RTX Pro 5000 · Distributed Muon · 监督微调（SFT）· 分支 `nccl-align-on-keddy`

> 在已有的视觉/对齐模块分片（improv1）和视觉位置处理优化（improv2）之上，开启 `--train.fsdp_align_root` 后，平均完整步时从 **4.2293 秒**降到 **4.2268 秒**，吞吐 **+0.06%**，与同一代码两次运行之间的波动（0.03%–0.05%）同一量级，可以认为没有收益。 原因是 improv1 已经把根单元缩小到每卡约 1.1 MB，未对齐只让根单元的 AllGather 每步多花约 2 ms。 同时复现了 improv1、improv2 相对官方基线的收益：**+5.23%**、**+6.60%**，与原报告（+5.11%、+6.52%）一致。

结论

- **对齐在当前代码上没有可测收益，不建议作为提速项交付。**改动默认关闭、不改变模型和数值，保留为可选开关即可。
- **问题本身存在，但已被 improv1 化解。**官方代码里根单元每卡约 239 MB，同样的未对齐会让根单元 AllGather 慢约 5 倍；improv1 把视觉编码器等移出根单元后，未对齐的影响降到每步约 2 ms。
- **官方基线、improv1、improv2 都已复现**，绝对步时比原报告快约 0.2%，相对收益差在 0.12 个百分点以内。

测试条件

项目	设置
硬件与数据	8 张 RTX Pro 5000（同一台机器）；RoboTwin 2.0 clean 50 任务，按官方 RoboTwin pi0 流程转换的图像数据（549,787 帧）
批量与优化器	micro batch 32；梯度累积 1；global batch 256；Distributed Muon；激活检查点；FSDP 前向后参数驻留
NCCL	`NCCL_MIN_CTAS=8`、`NCCL_NTHREADS=128`、`NCCL_P2P_READ_ENABLE=1`；RDMA 可用
运行环境	conda，PyTorch 2.8.0+cu128、Transformers 4.57.3、NCCL 2.27.3；每个 rank 绑定到 GPU 所在 NUMA 节点的 CPU
测量	每次 31 步，第 1–10 步预热，统计第 11–30 步；每步从取数据前到打完日志，两端 CUDA 同步，取 8 个 rank 的最大值。每次运行前 GPU 降到 40°C 以下，最高 64°C，无降频
运行顺序	基线、improv1、improv2、improv2、improv1、基线、improv2、对齐、对齐、improv2（交替，每版至少两次）

复现：官方基线、improv1、improv2

配置	commit	本次步时（秒）	原报告步时（秒）	本次相对基线	原报告相对基线	显存峰值（MiB）

官方基线	`be969b8`	4.5080	4.5107 / 4.5128	—	—	71,799
improv1：视觉、词嵌入、对齐模块分片与预取	`b55ea4a` + `412b978`	4.2840	4.2915	+5.23%	+5.11%	63,451
improv2：视觉位置处理优化	`42e963e`	4.2288	4.2366	+6.60%	+6.52%	63,507

提升 = 基线步时 ÷ 优化步时 − 1。基线为 2 次运行平均，improv1 为 2 次，improv2 为 4 次；同一版本各次运行之间相差不超过 0.05%，单次运行的步时变异系数 0.08%–0.17%。improv2 相对 improv1 的增量为 +1.30%（原报告 +1.29%）。官方基线的显存峰值与原报告完全一致。

对齐优化的原理与实现

- **原因**：FSDP2 把一个单元里各参数的分片拼成一块缓冲区做 AllGather，rank r 的数据从 r × 块大小处开始。NCCL 用 16 字节宽的指令搬数据，块大小不是 16 字节的倍数时会退回窄指令。
- **来源**：`model.action_out_proj.bias` 形状 (55,)，每卡分到 7 个元素，使根单元每卡的 bf16 块为 1,131,022 字节，除以 16 余 14。根单元里其他参数都是对齐的。
- **修复**：`--train.fsdp_align_root` 在对根单元调用 `fully_shard` 之前，找出每卡分片不是 8 个元素倍数的参数，把它所属的模块单独做成一个 FSDP2 单元。这里只有 `action_out_proj`（42,295 个参数）。默认关闭；参数名、形状和训练数学都不变。

性能结果

配置	第一次（秒）	第二次（秒）	平均（秒）	相对 improv2
improv2	4.2294	4.2291	4.2293	—
improv2 + 对齐	4.2279	4.2256	4.2268	+0.06%

表中 improv2 为与对齐交替运行的两次（顺序 improv2、对齐、对齐、improv2）；之前的两次 improv2 为 4.2280、4.2287 秒。

通信层面（每个 rank 记录每次集合通信的大小和耗时，5 步，rank 0，去掉第 1 步）：

集合通信	improv2	improv2 + 对齐
根单元前向 AllGather（bf16）	1,131,022 B，未对齐，约 3.3 ms	1,120,256 B，已对齐，约 1.0 ms
根单元 ReduceScatter（fp32）	2,262,044 B，未对齐，约 1.7 ms	2,240,512 B，已对齐，约 1.5 ms
新增 `action_out_proj` 单元	—	10,766 B，含等待时间

对齐后根单元 AllGather 快了约 3 倍，但它本身每步只有几毫秒，再加上新增小单元的开销，端到端差别落在测量波动内。作为对照，在官方代码的分片方式下根单元每卡约 239 MB，同样的修复能让每步快约 0.33 秒（此前在默认 NCCL 设置下测得）；improv1 把视觉编码器、词嵌入和对齐模块移出根单元后，这部分收益已经包含在 improv1 里了。

数值验证

对比（第 1–30 步，逐步相对差的平均 / 最大）	总 loss	VLA loss	grad norm	第 1 步
基线 vs 基线（同一代码，波动参考）	0.11% / 0.28%	0.13% / 0.38%	1.8% / 5.0%	全部相同
improv2 vs improv2（同一代码）	0.14% / 0.53%	0.16% / 0.56%	1.4% / 5.4%	全部相同
improv2 vs 对齐（第一对）	0.11% / 0.33%	0.15% / 0.47%	2.0% / 10.4%	loss 相同；grad norm 0.97446 vs 0.97300
improv2 vs 对齐（第二对）	0.17% / 0.47%	0.16% / 0.47%	2.0% / 10.9%	loss 相同；grad norm 同上

- 第 1 步的五项 loss（总 loss、VLA、深度、未来深度、未来视频）逐位相同，说明前向计算不变。
- 第 1 步 grad norm 相差 0.15%：拆出新单元后，梯度的 ReduceScatter 分组和 fp32 求和顺序变了。improv1 相对基线也有同样现象（0.9716 vs 0.9833）。之后各步的 loss 差异与同一代码两次运行之间的差异同一量级。
- 本结果是短测，没有验证长期收敛。